

Radiant College, Manigram Butwal

[ITEX 103] Object Oriented Programming
B Tech Ed. – 2ND SEMESTER

Unit - 2
(Object and Classes)

Objective:

1. Understand the basics of classes and objects
2. Be able to initialize objects by their class
3. Implement the use of **this** operator
4. Understand the idea of access specifiers
5. Understand and implement the use of constructors and destructors in C++

Classes and Objects

Class:

A class is a user-defined data type that serves as a blueprint for creating objects.

Object:

: An instance of a class (eg, Book myBook).

Example:

```
#include <iostream>
#include <string>

#include <iostream>
#include <string>
using namespace std;
```

```
// Class Definition
class Book {
private:    // Data members (attributes)
    string title;
    string author;
    double price;

public:     // Member functions (methods)
    // Setter functions
    void setTitle(string t) { title = t; }
    void setAuthor(string a) { author = a; }
    void setPrice(double p) { price = p; }

    // Getter functions
    string getTitle() { return title; }
    string getAuthor() { return author; }
    double getPrice() { return price; }

    // Utility function
    void display() {
        cout << "Book: " << title << " by " << author
            << "\nPrice: $" << price << endl;
    }
};

int main() {
    // Create object
    Book myBook;

    // Set data using member functions
    myBook.setTitle("The C++ Programming Language");
    myBook.setAuthor("Bjarne Stroustrup");
    myBook.setPrice(59.99);

    // Access data using member functions
    cout << "Title: " << myBook.getTitle() << endl;

    // Display complete book info
    myBook.display();

    return 0;
}
```

Explanation:

- Class Definition: `Book` class encapsulates data (title, author, price) and operations
- Data Members: Private variables storing object state
- Member Functions: Public methods to interact with private data
- Encapsulation: Data is protected, access controlled via methods

Access Specifiers

Access specifiers control how the members (attributes and methods) of a class can be accessed.

They help protect data and organize code so that only the right parts can be seen or changed.

There are three types of access specifiers:

- **Private :**
 - Members are accessible only within the class (default).
- **Public**
 - *Members are accessible anywhere.*
- **Protected**
 - Similar to `private` but accessible in derived classes (inheritance)

```
#include <iostream>
using namespace std;

class AccessDemo {
public:    // Accessible anywhere
    int publicVar = 10;
```

```
void publicMethod() {
    cout << "Public method called\n";
    cout << "Accessing privateVar: " << privateVar << endl; // Can
access private members
}

private: // Accessible only within class
    int privateVar = 20;

    void privateMethod() {
        cout << "Private method called\n";
    }

protected: // Accessible in class and derived classes
    int protectedVar = 30;
};

int main() {
    AccessDemo obj;

    // Public access (allowed)
    cout << "Public variable: " << obj.publicVar << endl;
    obj.publicMethod();

    // Private access (compiler error)
    // cout << obj.privateVar;
    // obj.privateMethod();

    // Protected access (compiler error in non-derived class)
    // cout << obj.protectedVar;

    return 0;
}
```

Explanation

- Public: Accessible anywhere
- Private: Accessible only within class
- Protected: Accessible in class and derived classes

Initializing Objects

```
#include <iostream>
using namespace std;

class Student {
private:
    string name;
    int id;
    double gpa;

public:
    // Initialization using member functions
    void initialize(string n, int i, double g) {
        name = n;
        id = i;
        gpa = g;
    }

    void display() {
        cout << "Student: " << name << "\nID: " << id
            << "\nGPA: " << gpa << endl;
    }
};

int main() {
    Student s1;
    s1.initialize("Alice Johnson", 1001, 3.8);

    Student s2;
    s2.initialize("Bob Smith", 1002, 3.5);

    cout << "First student:\n";
    s1.display();

    cout << "\nSecond student:\n";
    s2.display();

    return 0;
}
```

Explanation

- Objects start with uninitialized data
- Explicit initialization required before use
- Initialization methods provide controlled setup
- Ensures objects begin in valid state

Static

Static Data Members:

Static data members are class members that are declared using **static** keywords.

A static member has certain special characteristics which are as follows:

- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- It is initialized before any object of this class is created, even before the main starts outside the class itself.
- It is visible and can be controlled with the class access specifiers.
- Its lifetime is the entire program.

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
private:
    string name;
    int id;
```

```

// Static data member (class-level counter)
static int totalEmployees; // Declaration

public:
    // Constructor
    Employee(string n, int i) : name(n), id(i) {
        totalEmployees++; // Increment when object created
    }

    // Destructor
    ~Employee() {
        totalEmployees--; // Decrement when object destroyed
    }

    // Static member function to access static data
    static int getTotalEmployees() {
        return totalEmployees;
    }

    void display() {
        cout << "ID: " << id << " | Name: " << name << endl;
    }
};

// Initialize static member (MANDATORY)
int Employee::totalEmployees = 0;

int main() {
    cout << "Initial employee count: "
        << Employee::getTotalEmployees() << "\n\n";

    // Create employees
    Employee e1("Alice Johnson", 1001);
    Employee e2("Bob Smith", 1002);

    cout << "After hiring 2 employees:\n";
    e1.display();
    e2.display();
    cout << "Total employees: "

```

```
<< Employee::getTotalEmployees() << "\n\n";

{
    // Nested scope
    Employee e3("Charlie Brown", 1003);
    cout << "--- Inside block ---\n";
    cout << "New hire: ";
    e3.display();
    cout << "Total employees: "
        << Employee::getTotalEmployees() << "\n";
    cout << "--- Exiting block ---\n";
} // e3 destroyed here

cout << "\nAfter Charlie leaves:\n";
cout << "Total employees: "
    << Employee::getTotalEmployees() << endl;

return 0;
} // e1 and e2 destroyed here
```

Output:

```
Initial employee count: 0

After hiring 2 employees:
ID: 1001 | Name: Alice Johnson
ID: 1002 | Name: Bob Smith
Total employees: 2

--- Inside block ---
New hire: ID: 1003 | Name: Charlie Brown
Total employees: 3
--- Exiting block ---

After Charlie leaves:
Total employees: 2
```


Explanation

- Static Declaration:

```
static int totalEmployees; // Inside class
```

Declares `totalEmployees` as static (shared across all objects)

- Mandatory Initialization:

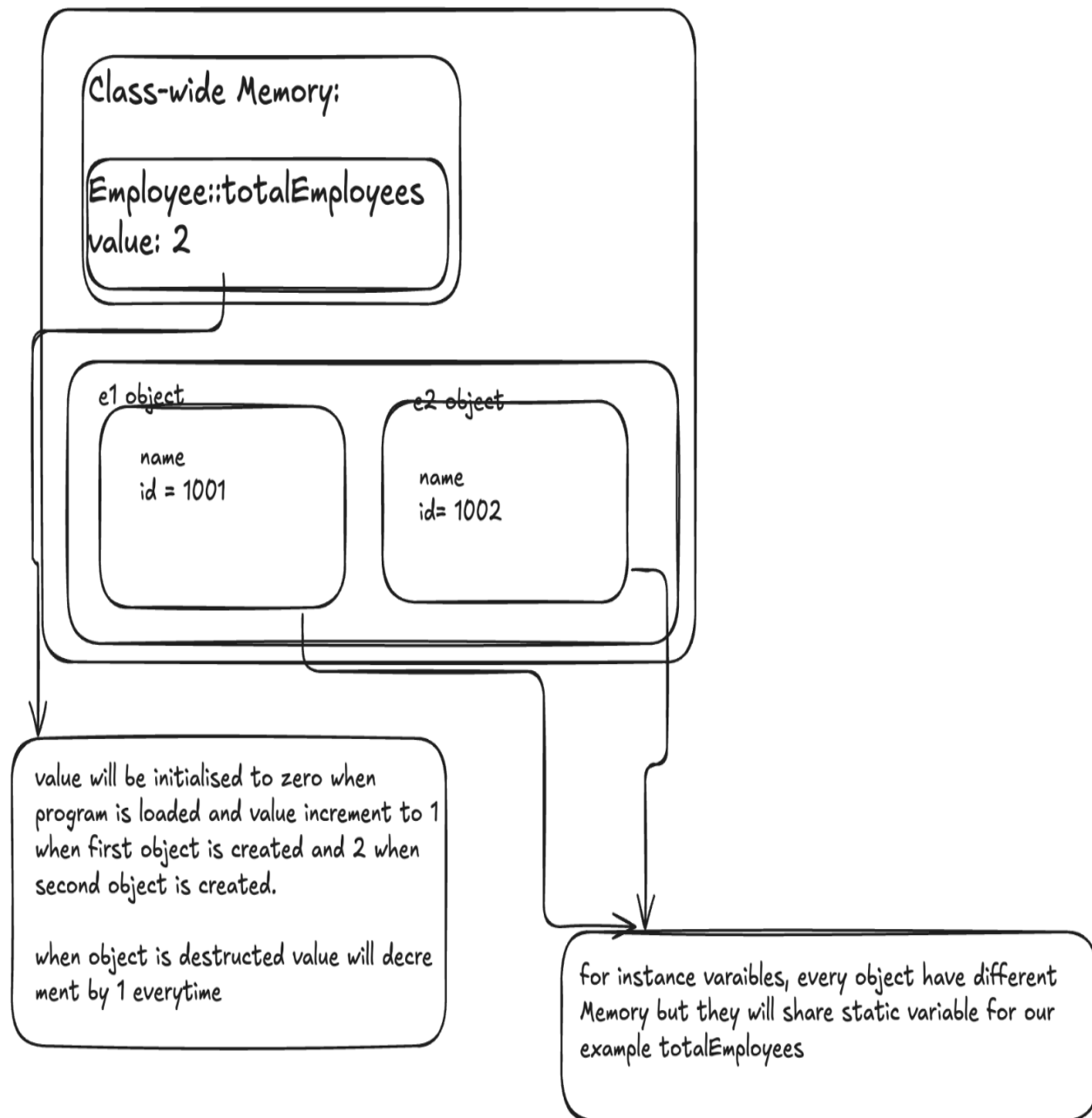
```
Int Employee:: totalEmployee =0;
```

Static Members must be initialized separately
Allocated in global memory space

- Object Creation/Destruction

```
Employee( ... ) { totalEmployee++; }  
  
~Employee( ) { totalEmployee - }
```

Increments counter when object is created
Decrements counter when object is destroyed



Static Member Functions:

A static member function is a function that belongs to a class, rather than an

object of the class.

```
cout << "Total employees: "  
      << Employee::getTotalEmployees() << "\n";  
cout << "--- Exiting block ---\n";
```

Explanation:

- **Class-level Functions:** Called using class name (::)
- **No Object Needed:** Exist independently of instances
- **Access Restrictions:** Can only access static members
- **Utility Functions:** Perfect for math operations/formatters

This Operator

Definition:

`this` is an implicit pointer available within non-static member functions of a class. It points to the *current object instance* invoking the function.

Syntax:

```
this->member_name; // Access members  
(*this).member_name; // Alternative syntax
```

Example Program:

```
#include <iostream>
using namespace std;

class Box {
    int width;
public:
    void setWidth(int width) {
        this->width = width; // Resolves name conflict
    }
    void print() {
        cout << "Width: " << this->width << endl;
    }
};

int main() {
    Box b;
    b.setWidth(5);
    b.print(); // Output: Width: 5
    return 0;
}
```

Explanation:

- `this` resolves ambiguity between class members and function parameters (e.g., `width` in `setWidth()`).
- It points to the object that called the function (`b` in this case).
- Required when:
 - Parameter names shadow member variables.
 - Returning the current object (e.g., `return *this;` in method chaining)

Constructor

Definition:

A special member function that ***automatically initializes*** objects when they are created.

It has the same name as the class and no return type.

Syntax:

```
ClassName(parameters) {  
    // Initialization code  
}
```

Example:

```
#include <iostream>
using namespace std;

class Student {
    string name;
    int age;
public:
    // Parameterized Constructor
    Student(string n, int a) : {
        name = n;
        age = a;
    } // Initialization list

    void display() {
        cout << name << ", " << age << " years old" << endl;
    }
};

int main() {
    Student s("Alice", 20); // Constructor called
    s.display(); // Output: Alice, 20 years old
    return 0;
}
```

Explanation:

- Invoked Automatically when an object is created.
- Types:
 - **Default Constructor**

Definition: A constructor that takes *no parameters* and provides default initialization for objects. If no constructors are defined, the compiler automatically generates a default constructor.

Syntax:

```
ClassName(  
    {  
        // initialization code (optional)  
    }
```

Example Program

```
#include <iostream>  
using namespace std;  
  
class Student {  
    string name;  
    int age;  
public:  
    // Default Constructor  
    Student() {  
        name = "Unknown";  
        age = 18;  
        cout << "Default constructor called!" << endl;  
    }  
  
    void display() {  
        cout << name << ", " << age << " years old" <<  
endl;  
    }  
};  
  
int main() {  
    Student s1;           // Calls default constructor  
    s1.display();         // Output: Unknown, 18 years old  
  
    Student s2();         // ⚠ Warning: Function
```

```
declaration (not object)
    Student s3{};           // ✓ Correct (C++11 uniform
initialization)
    return 0;
}
```

Explanation:

1. Automatically invoked when objects are declared *without arguments* (Student s1;).
2. Used to:
 - a. Set default values (e.g., name = "Unknown").
 - b. Allocate memory/resources.
 - c. Initialize internal state.
3. Compiler-generated default constructor:
 - a. Exists only if *no user-defined constructors* are declared.
 - b. Does *not* initialize primitive types (e.g., int age contains garbage value).

○ Parameterized Constructor

Syntax:

```
ClassName(Type1 param1, Type2 param2, ...) {
    // Initialization code
}
```

Example Program:


```
#include <iostream>
using namespace std;

class Point {
    int x, y;
public:
    // Parameterized Constructor
    Point(int xVal, int yVal) : {
        x=xVal;
        y=yVal;
    } // Initialization list

    void display() {
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Point p1(3, 4);    // Direct call
    p1.display();      // Output: (3, 4)

    Point p2 = Point(5, 6); // Explicit call
    p2.display();      // Output: (5, 6)
    return 0;
}
```

Explanation:

1. Allows object initialization with specific values at creation time.

- Copy Constructor

Definition:

A constructor that creates a new object as a copy of an existing object. It takes a reference to an object of the same class as its parameter.

Syntax:

```
ClassName(const ClassName &obj) {  
    // Copy member values  
}
```

Example Program:

```
#include <iostream>  
#include <string>  
using namespace std;  
  
class Student {  
    string name;  
    int age;  
public:  
    // Parameterized Constructor (assignment in body)  
    Student(string n, int a) {  
        name = n;  
        age = a;  
    }  
  
    // Copy Constructor (assignment in body)  
    Student(const Student &other) {  
        name = other.name;  
        age = other.age;  
        cout << "Copy constructor called!" << endl;  
    }  
  
    void display() {  
        cout << name << ", " << age << " years old" << endl;  
    }  
}
```

```

    }
};

int main() {
    Student s1("Alice", 20);
    cout << "Original: ";
    s1.display();

    Student s2 = s1;
    cout << "Copy: ";
    s2.display();

    return 0;
}

```

Key Points:

1. Copy Constructor Signature:

```
Student(const Student &other)
```

- a. `const`: Promises not to modify the original object
- b. `&other`: Reference to the object being copied

2. Member Initialization:

```

Student(const Student &other) {
    name = other.name;
    age = other.age;
    cout << "Copy constructor called!" << endl;
}

```

Directly copies values from `other` to the new object's members

3. Automatic Invocation:

- a. The copy constructor is called automatically during:

```

Student s2 = s1;    // Explicit copy initialization
Student s3(s1);     // Direct initialization

```

Constructor Overloading:

Definition:

Constructor overloading is a technique where a class defines multiple constructors with different parameter lists. This allows objects to be initialized in various ways, providing flexibility and convenience. The compiler selects the appropriate constructor based on the arguments passed during object creation.

Key Characteristics:

1. Same name as the class
2. Different numbers or types of parameters
3. No return type (not even void)
4. Can have different access specifiers (though typically public)

Example:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    string name;
    int age;
    float gpa; // New member for demonstration
public:
    // Constructor 1: Default Constructor
    Student() {
        name = "Unknown";
        age = 18;
        gpa = 0.0;
        cout << "Default constructor called" << endl;
    }

    // Constructor 2: Name-only Constructor
    Student(string n) {
```

```

        name = n;
        age = 18;           // Default age
        gpa = 0.0;          // Default GPA
        cout << "Name-only constructor called" << endl;
    }

    // Constructor 3: Name and Age Constructor
    Student(string n, int a) {
        name = n;
        age = a;
        gpa = 0.0;          // Default GPA
        cout << "Name+Age constructor called" << endl;
    }

    // Constructor 4: Full Details Constructor
    Student(string n, int a, float g) : name(n), age(a), gpa(g) {
        cout << "Full-details constructor called" << endl;
    }

    // Constructor 5: Copy Constructor
    Student(const Student &other) {
        name = other.name;
        age = other.age;
        gpa = other.gpa;
        cout << "Copy constructor called" << endl;
    }

    void display() {
        cout << name << ", " << age << " years, GPA: " << gpa <<
endl;
    }
};

int main() {
    // Different ways to create Student objects
    Student s1;           // Default constructor
    Student s2("Alice");   // Name-only constructor
    Student s3("Bob", 20); // Name+Age constructor
    Student s4("Charlie", 22, 3.8); // Full-details constructor

```

```
Student s5 = s4;                // Copy constructor

s1.display(); // Unknown, 18 years, GPA: 0
s2.display(); // Alice, 18 years, GPA: 0
s3.display(); // Bob, 20 years, GPA: 0
s4.display(); // Charlie, 22 years, GPA: 3.8
s5.display(); // Charlie, 22 years, GPA: 3.8

return 0;
}
```

Key Points

1. Parameter Differentiation:

Constructors are distinguished by their parameter lists:

- a. `Student()` → 0 parameters
- b. `Student(string)` → 1 parameter
- c. `Student(string, int)` → 2 parameters
- d. `Student(string, int, float)` → 3 parameters
- e. `Student(const Student&)` → Copy constructor signature

2. Constructor Selection:

The compiler chooses the constructor based on initialization:

```
Student s1;                // → Default (0 params)
Student s2("Alice");       // → string param
Student s3("Bob", 20);     // → string + int params
```

Destructor in C++

Definition:

A destructor is a special member function that automatically executes when an object goes out of scope or is explicitly destroyed. Its primary purpose is to release resources acquired by the object (memory, files, network connections) to prevent leaks.

Key Characteristics:

- Name: `~ClassName()`
- No parameters or return type
- Automatically invoked
- Each class has exactly one destructor
- Cannot be overloaded or called explicitly

Syntax

```
~ClassName() {  
    // Cleanup code (e.g., delete, close, release)  
}
```

Example:

```
#include <iostream>  
#include <string>  
using namespace std;  
  
class Student {  
    string name;  
    int age;  
public:  
    // Constructor  
    Student(string n, int a) : name(n), age(a) {  
        cout << name << " created!" << endl;  
    }  
  
    // Destructor (simplified)
```

```
~Student() {  
    cout << name << " destroyed!" << endl;  
}  
  
void display() {  
    cout << name << ", " << age << " years old" << endl;  
}  
};  
  
int main() {  
    cout << "Main starts" << endl;  
  
    { // Start of nested scope  
        cout << "\nEntering nested scope..." << endl;  
        Student alice("Alice", 20);  
        alice.display();  
        cout << "Leaving nested scope..." << endl;  
    } // alice destroyed here  
  
    cout << "\nBack in main" << endl;  
  
    Student* bob = new Student("Bob", 21);  
    bob->display();  
    delete bob; // Explicit destruction  
  
    cout << "\nMain ends" << endl;  
    return 0;  
}
```